

```

#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;

struct Node {
    string name;
    long long int phone_number;
    Node* next;
    Node* prev;
};

class ContactBook {
    Node* head;
    string x;
    long long int y;

public:
    ContactBook() {
        head = nullptr;
        x = "";
        y = 0;
    }

    void CreateNode() {
        Node* newer = new Node;
        cout << " Name of Contact: ";
        cin >> ws; // Clean whitespace
        getline(cin, x);
        newer->name = x;

        cout << " Phone Number: ";
        cin >> y;
        newer->phone_number = y;
        newer->next = nullptr;
        newer->prev = nullptr;

        if (head == nullptr) {
            head = newer;
        } else {
            Node* temp = head;
            while (temp->next != nullptr)
                temp = temp->next;
            temp->next = newer;
        }
    }
};

```

```

        newer->prev = temp;
    }
    cout << " Contact Added Successfully\n";
}

void Display() {
    Node* temp = head;
    int count = 0;

    if (temp == nullptr) {
        cout << " No Contacts... Please Add Some Contacts\n";
        return;
    }

    BubbleSort();
    cout << " Name:          Number:\n";
    while (temp != nullptr) {
        count++;
        cout << " " << temp->name << "    " << temp->phone_number << "\n";
        temp = temp->next;
    }
    cout << " Total contacts: " << count << "\n";
}

int Search() {
    Node* temp = head;
    bool found = false;
    int command;
    cout << " 1. Search by Name\n";
    cout << " 2. Search by Number\n";
    cout << " Enter your choice: ";
    cin >> command;

    if (command == 1) {
        cout << " Enter the Name to Search: ";
        cin >> ws;
        getline(cin, x);
        while (temp != nullptr) {
            if (temp->name == x) {
                cout << " Name: " << temp->name << "\n";
                cout << " Number: " << temp->phone_number << "\n";
                found = true;
                break;
            }
        }
    }
}

```

```

        temp = temp->next;
    }
} else if (command == 2) {
    cout << " Enter the Number to Search: ";
    cin >> y;
    while (temp != nullptr) {
        if (temp->phone_number == y) {
            cout << " Name: " << temp->name << "\n";
            cout << " Number: " << temp->phone_number << "\n";
            found = true;
            break;
        }
        temp = temp->next;
    }
}

if (!found)
    cout << " Contact not found.\n";

return 0;
}

```

```

int DeleteContactBySearch() {
    Node* temp = head;
    bool found = false;
    int command;
    cout << " 1. Delete by Name\n";
    cout << " 2. Delete by Number\n";
    cout << " Enter your choice: ";
    cin >> command;

    if (command == 1) {
        cout << " Enter the Name to Delete: ";
        cin >> ws;
        getline(cin, x);
        while (temp != nullptr) {
            if (temp->name == x) {
                found = true;
                break;
            }
            temp = temp->next;
        }
    } else if (command == 2) {
        cout << " Enter the Number to Delete: ";
    }
}

```

```

    cin >> y;
    while (temp != nullptr) {
        if (temp->phone_number == y) {
            found = true;
            break;
        }
        temp = temp->next;
    }
}

if (found) {
    if (temp == head) {
        head = temp->next;
        if (head) head->prev = nullptr;
    } else {
        if (temp->prev) temp->prev->next = temp->next;
        if (temp->next) temp->next->prev = temp->prev;
    }
    delete temp;
    cout << " Contact Deleted Successfully.\n";
} else {
    cout << " Contact Not Found.\n";
}

return 0;
}

```

```

int EditContacts() {
    Node* temp = head;
    bool found = false;
    int command;
    cout << " 1. Edit by Name\n";
    cout << " 2. Edit by Number\n";
    cout << " Enter your choice: ";
    cin >> command;

    if (command == 1) {
        cout << " Enter the Name to Edit: ";
        cin >> ws;
        getline(cin, x);
        while (temp != nullptr) {
            if (temp->name == x) {
                found = true;
                break;
            }
        }
    }
}

```

```

        }
        temp = temp->next;
    }
} else if (command == 2) {
    cout << " Enter the Number to Edit: ";
    cin >> y;
    while (temp != nullptr) {
        if (temp->phone_number == y) {
            found = true;
            break;
        }
        temp = temp->next;
    }
}

if (found) {
    cout << " Enter New Name: ";
    cin >> ws;
    getline(cin, temp->name);
    cout << " Enter New Number: ";
    cin >> temp->phone_number;
    cout << " Contact Edited Successfully.\n";
} else {
    cout << " Contact Not Found.\n";
}

return 0;
}

void DeleteAllContacts() {
    Node* temp = head;
    while (temp != nullptr) {
        Node* next = temp->next;
        delete temp;
        temp = next;
    }
    head = nullptr;
    cout << " All Contacts Deleted.\n";
}

void BubbleSort() {
    if (!head) return;
    for (Node* i = head; i->next != nullptr; i = i->next) {
        for (Node* j = i->next; j != nullptr; j = j->next) {

```

```

        if (i->name > j->name) {
            swap(i->name, j->name);
            swap(i->phone_number, j->phone_number);
        }
    }
}

```

```

void OfflineSave() {
    ofstream myfile("contactbook.txt");
    Node* temp = head;
    while (temp != nullptr) {
        myfile << temp->name << "\n";
        myfile << temp->phone_number << "\n";
        temp = temp->next;
    }
    myfile.close();
}

```

```

void reopenCB() {
    ifstream myfile("contactbook.txt");
    if (!myfile.is_open() || myfile.peek() == EOF) {
        return;
    }
}

```

```

string name, numberLine;
long long int number;

```

```

while (getline(myfile, name) && getline(myfile, numberLine)) {
    stringstream ss(numberLine);
    ss >> number;
}

```

```

Node* newer = new Node;
newer->name = name;
newer->phone_number = number;
newer->next = nullptr;
newer->prev = nullptr;

```

```

if (head == nullptr) {
    head = newer;
} else {
    Node* temp = head;
    while (temp->next != nullptr)
        temp = temp->next;
}

```

```

        temp->next = newer;
        newer->prev = temp;
    }
}

myfile.close();
}

void Structure() {
    int Scommand;
    while (true) {
        cout << "\n*****\n";
        cout << " 1. Add Contact\n";
        cout << " 2. Edit Contact\n";
        cout << " 3. Delete Contact\n";
        cout << " 4. Search Contact\n";
        cout << " 5. Display All Contacts\n";
        cout << " 6. Delete All Contacts\n";
        cout << " 7. Exit\n";
        cout << "*****\n";
        cout << " Enter your choice: ";
        cin >> Scommand;

        switch (Scommand) {
            case 1: CreateNode(); break;
            case 2: EditContacts(); break;
            case 3: DeleteContactBySearch(); break;
            case 4: Search(); break;
            case 5: Display(); break;
            case 6: DeleteAllContacts(); break;
            case 7:
                OfflineSave();
                cout << " Exiting ContactBook. Goodbye!\n";
                return;
            default:
                cout << " Invalid command. Try again.\n";
        }

        OfflineSave(); // Save after each operation
    }
}

};

int main() {

```

```
ContactBook cb;
cb.reopenCB();
string user;
cout << " What is your name: ";
cin >> ws;
getline(cin, user);

cout << "\n*****\n";
cout << " Welcome, " << user << " to ContactBook\n";
cout << "*****\n";
cb.Structure();
return 0;
}
```